

CAPSTONE PROJECT REPORT

(Project Term August-December 2025)

AI-DRIVEN PUBLIC HEALTH CHATBOT FOR DISEASE AWARENESS

(Smart India Hackathon - PS Number: SIH25049)

Submitted by

Gutha Sahith Chowdary Registration Number: 12201291

Sai Hrieshikesh Elluru Registration Number: 12216124

Loka Manikanta Neeluri Registration Number: 12210714

Patlolla Varshith Reddy Registration Number: 12201442

Kamadula Siddartha Registration Number: 12200271

Vutti Ajay Registration Number: 12200245

Project Group Number: KC1

Course Code: CSE339

Under the Guidance of

Arshiya

Associate Professor, Department of CSE

School of Computer Science and Engineering

Lovely Professional University, Phagwara

PAC FORM

(The Project Approval Committee form duly signed by the panel members and the mentor is to be attached at this position. This page is reserved for the official PAC documentation issued by the School of Computer Science and Engineering.)

Project Title: AI-Driven Public Health Chatbot for Disease Awareness

Project Group Number: KC1

Course Code: CSE339

Mentor: Arshiya, Associate Professor, Department of CSE

Approved by:

Panel Member 1

Panel Member 2

Mentor

DECLARATION

We hereby declare that the project work entitled “AI-Driven Public Health Chatbot for Disease Awareness” is an authentic record of our own work carried out as the requirement of the Capstone Project for the award of the B.Tech degree in Computer Science and Engineering from Lovely Professional University, Phagwara, under the guidance of Arshiya, Associate Professor, Department of CSE, during the project term August–December 2025. All the information furnished in this capstone project report is based on our own intensive work and is genuine.

Project Group Number: KC1

Name of Student 1: Gutha Sahith Chowdary

Registration Number: 12201291

Name of Student 5: Kamadula Siddartha

Registration Number: 12200271

Name of Student 6: Vutti Ajay

Registration Number: 12200245

(Signature of Student 1)

Date:

(Signature of Student 2)

Date:

(Signature of Student 3)

Date:

CERTIFICATE

This is to certify that the declaration statement made by this group of students is correct to the best of my knowledge and belief. They have completed this Capstone Project under my guidance and supervision. The present work is the result of their original investigation, effort, and study. No part of the work has ever been submitted for any other degree at any university. The Capstone Project is fit for submission and partial fulfilment of the conditions for the award of the B.Tech degree in Computer Science and Engineering from Lovely Professional University, Phagwara.

Signature and Name of the Mentor

Arshiya

Associate Professor

Department of Computer Science and Engineering

School of Computer Science and Engineering,

Lovely Professional University, Phagwara, Punjab.

Date:

ACKNOWLEDGEMENT

The completion of this Capstone Project would not have been possible without the support, guidance, and encouragement of many individuals to whom we owe sincere gratitude.

First and foremost, we extend our deepest thanks to our project mentor, Arshiya, Associate Professor, Department of Computer Science and Engineering, Lovely Professional University. Her steady supervision, patient feedback, and willingness to engage with our questions at every stage shaped the technical direction of this project and helped us recover whenever we ran into design dead-ends. Many of the architectural decisions described in this report were refined during her review sessions.

We are equally grateful to the faculty members of the School of Computer Science and Engineering for the foundational courses that prepared us for a project of this scale. The cumulative knowledge gained across software engineering, database systems, web technologies, artificial intelligence, and human–computer interaction courses formed the backbone on which the entire system stands.

We would like to thank the Smart India Hackathon organising committee for framing the problem statement SIH25049, which gave our team the opportunity to work on a problem that has tangible social impact. The clarity of the problem brief allowed us to focus on building a usable system rather than searching for direction.

Our sincere appreciation also goes to our families, whose continuous encouragement gave us the confidence to take on the long development hours that this project demanded. Their patience during the late submissions and the frequent project discussions at home is acknowledged with heartfelt gratitude.

Finally, we thank our peers and seniors at Lovely Professional University for the informal code reviews, design suggestions, and the willingness to test early prototypes of the chatbot. Their honest feedback caught several usability issues that we would otherwise have missed.

Gutha Sahith Chowdary

Sai Hrieshikesh Elluru

Loka Manikanta Neeluri

Patlolla Varshith Reddy

Kamadula Siddartha

Vutti Ajay

TABLE OF CONTENTS

Inner First Page	i
PAC Form	ii
Declaration	iii
Certificate.....	iv
Acknowledgement	v
Table of Contents.....	vi
1. INTRODUCTION	1
1.1. Background and Motivation	1
1.2. Objectives.....	2
1.3. Project Description.....	2
1.4. Scope.....	3
1.4.1. Use Case Model	3
2. PROFILE OF THE PROBLEM.....	5
2.1. Problem Statement (SIH25049).....	5
2.2. Rationale	6
3. EXISTING SYSTEM	7
3.1. Introduction.....	7
3.2. Existing Software.....	7
3.3. Data Flow Diagrams	8
3.4. What is New in the Proposed System.....	10
4. PROBLEM ANALYSIS.....	12
4.1. Product Definition.....	12
4.2. Feasibility Analysis.....	12
4.3. Project Plan	14
5. SOFTWARE REQUIREMENT ANALYSIS.....	16
5.1. Introduction.....	16
5.2. General Description	16

5.3. Specific Requirements	17
6. DESIGN.....	20
6.1. System Design	20
6.2. Notations Used.....	21
6.3. Detailed Design.....	22
6.4. Flowcharts.....	26
6.5. Pseudocode.....	27
7. TESTING.....	29
7.1. Functional Testing	29
7.2. Structural Testing	29
7.3. Levels of Testing.....	30
7.4. Test Summary	31
8. IMPLEMENTATION.....	33
8.1. Implementation of the Project.....	33
8.2. Conversion Plan	34
8.3. Post-Implementation and Maintenance.....	35
9. PROJECT LEGACY	36
9.1. Current Status.....	36
9.2. Remaining Areas of Concern.....	36
9.3. Lessons Learnt	37
9.4. Future Scope	38
10. USER MANUAL.....	40
11. SOURCE CODE / SNAPSHOTS.....	42
12. BIBLIOGRAPHY	45

1. INTRODUCTION

1.1 Background and Motivation

The Indian population is not necessarily health literate. There is a lot of social media misinformation, public health centres are the first points of contact for symptoms that need to be triaged electronically, and existing medical chatbots are either deterministic (FAQ) or unconstrained (open LLMs that hallucinate). The Smart India Hackathon (SIH25049) challenge is to build an AI chatbot that's all this: it's accurate, constrained, scalable and safe to recommend treatment.

We have a web app for deterministic triage, real-time AI video chat and follow up. Our solution is an integrated system on a modern TypeScript stack and managed cloud service, so it's a product, not a prototype. Our solution has three points of integration that target the safety aspects: deterministic system for triage, LLM agent for language chat with a narrow system prompt and asynchronous summariser for summarisation.

1.2 Objectives

1. Generate a triage chat with deterministic triage (self-care, AI triage or visit) of symptoms.
2. Provide an AI voice and video call that includes triage data and is "light".
3. Generate summaries (HPI, assessment and plan) of the phone calls and chat summaries.
4. Record user, agent, meeting and triage records with user segregation and audit.
5. Figure out how to link to doctors and hospitals.

1.3 Project Description

It's a Next.js 15 (App Router) application, with PostgreSQL (Neon) database. It has BetterAuth login (Google, GitHub, email/password). A user has their own AI doctor, and undergoes three phases:

- Phase 1, Triage Chat. A real-time (text) chat on chief complaint, symptom lexicon, location, severity and red-flags. A deterministic action is taken of STOP, GO_TO_DOCTOR or GO_TO_AI_AGENT.
- Phase 2, AI Video Consultation. A SFU room is created in Stream Video. An OpenAI Realtime agent is invited with a system message which is a combination of the meeting agent and triage agent payload. Recording and transcription is enabled.
- Phase 3, Post-Meeting Chat. The transcript is sent to Inngest pipeline that will summarise the transcript as a medical summary using GPT-4o. The user can chat with the model using the summary.

1.3.1 Technology Stack

Table 1.1: Core Technology Stack

Layer	Technology	Role
Framework	Next.js 15 App Router, React 19	RSC plus client routing
Language	TypeScript	Type safety end-to-end
API	tRPC v11 + TanStack Query v5	Type-safe RPC and caching
Database	PostgreSQL on Neon, Drizzle ORM	Persistence and migrations
Auth	BetterAuth	Multi-provider identity
Realtime Media	Stream Video and Chat SDKs	SFU, recording, transcription
AI Voice Agent	OpenAI Realtime API	WebRTC voice consultation
Summarisation	Inngest Agent Kit + GPT-4o	Background pipeline
UI	shadcn/ui + Tailwind CSS v4	Design system
Validation	Zod	Runtime schema checks

1.4 Scope

In scope: web triage, video consultation (AI), medical summarisation, history, authentication and isolation. Out of scope for this term: regulated diagnosis, prescription, multilingual UI, mobile-native apps, and direct integration with hospital information systems. These are tracked under future scope.

1.4.1 Use Case Model

Primary actor is the user. Other actors are an AI medical agent (system actor), the OpenAI Realtime, the Stream call SFU and Inngest job runner. The use cases are user sign up and sign in, create AI agent, triage chat, start AI consultation, video call, view summary, chat on summary, view history. They're mapped to a tRPC procedure or a stateful use case is mapped to a Stream webhook and an Inngest function. So the system is easy to design: the intent of the actor is mapped to a single server endpoint.

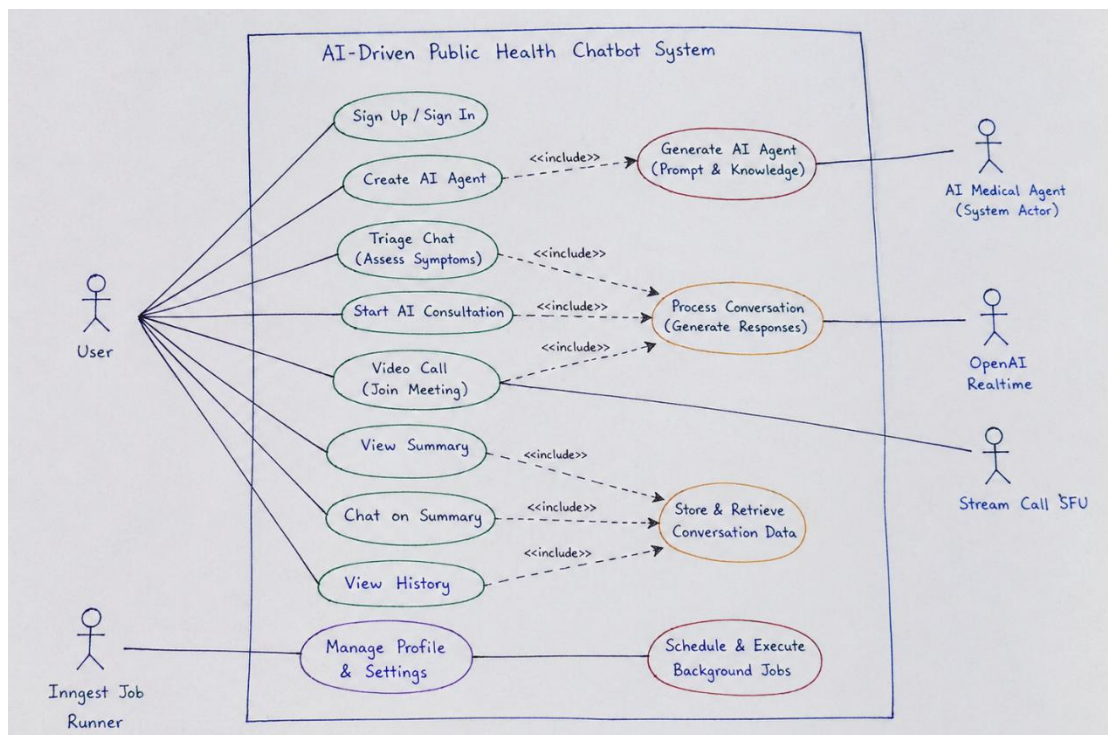


Figure 1.1: Use Case Diagram of the Public Health Chatbot

2. PROFILE OF THE PROBLEM

2.1 Problem Statement (SIH25049)

The problem statement is for a public health artificial intelligence (AI) chatbot that advises on diseases, does simple triage and operates at an epidemiology level. From an engineering perspective this means:

- Simple dialogue system without advice.
- The ability to escalate severe symptoms or symptoms that require further investigation.
- Text and speech to accommodate varying literacy.
- An interaction record for a clinician to review and for training. The requirements are: test the system to see if it can refer the real symptom report of a patient to the right level of care, provide an "interaction record" for a clinician and be cost effective as well as fast enough for a public service. The above four requirements are FR-1 to FR-11, Chapter 5.

2.2 Rationale

Open LLMs are not safe because they will answer anything Rules bots are not quick The split in this project is that the triage is rule-based (to ensure safety of the first decision); the consultation is conversational and uses an LLM with a constrained system prompt; the second pass (to summarise) is structured information. This is the technical contribution of this project.

The split has a second benefit: The triage engine is pure, and so can be tested with fixtures. The prompt builder is pure, so it can be snapshot tested. The LLM call is impure, but it's not controlled by the prompt and the summarisation. Errors are limited, are exposed and acceptable.

3. EXISTING SYSTEM

3.1 Introduction

There are four types of public health chatbots. They all cover some of the components of SIH25049 but no-one covers triage, real-time voice consultation and follow up in an audit trail. The argument below explains the solution.

3.2 Existing Software

3.2.1 Rule-Based Public Health Bots

These include COVIBOT (pandemic times), MoHFW WhatsApp bot and state dashboards. They are safe (all responses are string literals) but they are only FAQ bots. They are not adaptive to different ways of describing symptoms, do not triage and do not consult.

3.2.2 Telemedicine Platforms

Practo, 1mg, Tata Health have doctors to consult. They are real and safe but without AI triage; they can go astray. The price and queue size limit the market to the public-health market that SIH25049 seeks to engage.

3.2.3 General LLM Assistants

ChatGPT and Gemini are great conversationalists and capable of small talk about medicine. They lack specificity, context of the patient and have known hallucinations around drug doses, drug interactions and severity of symptoms. They are unsafe to use in public health without a scoping and auditing tool.

3.2.4 Academic Prototypes

HACI, some MedBots and some published research systems have intent classification and retrieval, sometimes with reinforcement learning for response selection. They often don't include voice consultation, seldom have production-ready user authentication and are not persistent. The new system has the same inclinations as these systems (retrieval vs generation) but is production-ready.

3.3 Data Flow Diagrams

The new system has two levels of DFD. The context DFD comprises of the user, the platform and the external services (Stream, OpenAI Realtime, Inngest, Neon) in a black box around the platform. The level-1 DFD shows the platform is made up of Auth, Triage Engine, Meeting Orchestrator, Summarisation Pipeline and Persistence. The data stores (user/session, agents, meetings, triageSessions) and the API (tRPC, webhook, REST) are shown. Our system is the only one that has deterministic triage, consultation by voice AI and summarisation of calls in an auditable process.

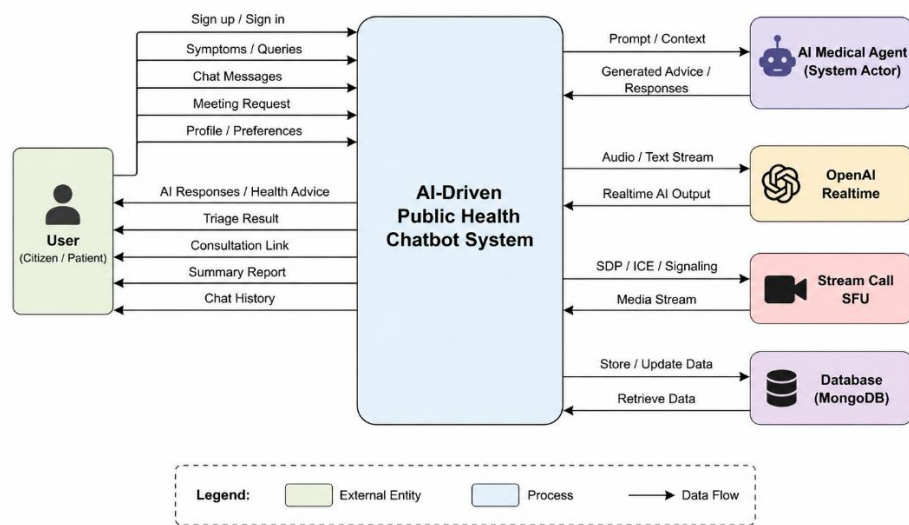


Figure 3.1: Context-Level Data Flow Diagram

[Insert Figure 3.2: Level-1 DFD here]



Figure 3.2: Level-1 Data Flow Diagram

3.4 What is New in the Proposed System

Table 3.1: Comparison of Existing Approaches with the Proposed System

Capability	Rule Bots	Telemedicine	Open LLMs	Proposed
Symptom Triage	Limited	Manual	Unsafe	Deterministic
AI Voice Consultation	No	Human only	Text only	Realtime API
Structured Summary	No	Manual notes	No	Inngest + GPT-4o
Per-User History	Partial	Yes	No	Yes (Drizzle/Neon)
Auditable Transcripts	No	Partial	No	Stream + DB
Scope Safety	High	High	Low	High (bounded prompt)
Cost per Session	Very Low	High	Low	Low to Moderate
Population-Scale Reach	High	Limited	High	High

The proposed "dual-architecture" system is safe like the first, auditable like the second, and fast like the third

4. PROBLEM ANALYSIS

4.1 Product Definition

The product is a web service that offers three primary services: (a) a predictable triage chatbot for information and treatment of diseases, (b) an AI-aided realtime video call, and (c) an automatically generated structured summary of the video call with searchable history. The product is a multi-tenant (user isolation) product that is delivered as a single package on Vercel + Neon.

The user interface is split up into four groups of React routes with the App Router: a public marketing site, a private dashboard, a triage stepper and a meeting room. The route groups have their own root layouts and loading boundaries so that we can flash up server-rendered shells and progressively load the client-only code (call interface, chat, triage stepper). This will allow us to achieve a low first-contentful-paint on instant pages, even on shared 4G networks (which our target users are on).

4.2 Feasibility Analysis

4.2.1 Technical Feasibility

The external services are sophisticated software-as-a-service (SaaS) products: Neon Postgres for the database, BetterAuth for authentication, Stream Video for call recording and an SFU (Selective Forwarding Unit), OpenAI Realtime for the voice of the AI agent and Inngest for jobs. The team has developed code to test the five integrations, so it is technically possible. The largest innovation, adding an OpenAI Realtime agent to the call is tested with every call.

4.2.2 Economic Feasibility

The cost to run the system is dominated by OpenAI Realtime, followed by Stream call minutes, and Neon storage. For hackathon, they are free or trial. At pilot scale (1,000 consultations per day) the cost per consultation is low and can be covered or passed through to partners. Rough estimate: realtime call (with average consultation length 8 minutes) costs less than \$0.50, a Stream call costs \$0.10 and summarisation (Inngest) is just a single call to GPT-4o (costs \$0.02). This is the basis for the conversion plan in Chapter 8.

4.2.3 Operational Feasibility

A web browser and microphone/ camera is required. No installation. The triage process falls back to text if media isn't permitted. The admin has a dashboard not a console. The deployment is a git push: Vercel builds, edge-caches and has immutable deployments so rollbacks are atomic.

4.2.4 Legal and Ethical Feasibility

It is only for triage, not diagnosis. It provides information about each step to the user. Data security is in accordance with India's IT Rules 2011 and DPDP Act 2023: user consent on registration, export and delete buttons for user, data in transit (TLS), and at rest (Neon). The AI agent is not allowed to want to be a doctor, and refers to the doctor if it moves into a black zone (prescription, dose or end of life).

4.3 Project Plan

Table 4.1: Project Phase Plan (August to December 2025)

Phase	Window	Deliverable
Requirements	Aug W1–W2	SRS, problem alignment with SIH25049
Architecture	Aug W3–W4	Module map, schema, integration choices
Auth + DB	Sep W1–W2	BetterAuth + Drizzle schema migrations
Agents + Meetings	Sep W3–Oct W1	tRPC modules, dashboard
Stream Video	Oct W2–W3	Realtime call + recording + transcript
Triage Engine	Oct W4–Nov W1	Symptom dictionary, decision engine, chat UI
Inngest Pipeline	Nov W2–W3	Webhook orchestrator, summarisation
Post-Meeting Chat	Nov W4	Stream Chat integration, scoped GPT-4o
Testing + Report	Dec W1–W2	Unit/integration tests, capstone report

The phases were not in strict sequence. By mid-September the schema and the auth were ready, so the agent, meetings and triage engine could be evolved on different Drizzle "migrations". The last critical path item was Inngest, as it requires data for training; this changed the order of integration, but not the order of phases.

5. SOFTWARE REQUIREMENT ANALYSIS

5.1 Introduction

This chapter outlines the functional (FRs) and non-functional requirements (NFRs) that were identified from SIH25049 and the design in Chapter 4. All requirements are testable and traceable to an implementation tRPC procedure, webhook event or user interface. All functional requirements have at least one test in Chapter 7.

5.2 General Description

The app provides services for logged in users, in the desktop (web) application and mobile app. It has a user space for Agents, Meetings and Triage. The server is event-driven: Stream webhooks for events that update server state for Inngest jobs; tRPC for event requests to and responses from React. There is a sidebar and three products (Agents, Meetings, Triage); and history table.

5.3 Specific Requirements

5.3.1 Functional Requirements

- FR-1: Create an account and register with Google, GitHub and/or email/password provider BetterAuth.
- FR-2: Create, read, update and delete AI medical agents (consist of name and instruction).
- FR-3: Create a meeting for one agent: meeting is scheduled.
- FR-4: Run a triage chat which asks chief complaint, symptom check list, location, severity and red-flag and makes a decision.
- FR-5: If decision is GO_TO_AI_AGENT, save triage data and go to meeting.
- FR-6: Invite the AI agent to join the Stream call with system message containing agent instructions and triage information.
- FR-7: Turn on recording and transcript on the events: session_started; recording_ready and recording_ready.
- FR-8: Asynchronously create the summary with the transcript via GPT-4o with Inngest.

- FR-9: Save the summary with the meeting; indicate meeting is finished.
- FR-10 Chat with the user with a model that has the summary.
- FR-11: Agent / meeting / triage history pages, list by user.
- **5.3.2 Non-Functional Requirements**
- NFR-1: P95 response of triage steps less than 500ms P95 response to chat token to 1st byte less than 1s.
- NFR-2: All database scoped with UserID. •
- NFR-3: All service (and other) tokens are server side; client has no secrets.
- NFR-4: All is secure at rest (Neon encryption) and in transit (TLS), client is idempotent, signed small tokens for Stream.
- NFR-5: Scalable; application service is stateless; Postgres, Stream and Inngest store all the data.
- NFR-6: Webhooks (handlers) idempotent (do nothing with duplicates).
- NFR-7: Summarisation will be retried 3 times (with exponential back-off) maximum and if it fails will cancel meeting.
- NFR-8: Document will be in English (Class-8 level), Hindi to be added.

5.3.3 Hardware and Software Interfaces

- Client: any browser with WebRTC, microphone and camera.
- Server: Node.js 20 (Vercel serverless functions)
- APIs: Stream Video and Chat REST and SDK, OpenAI Realtime via WebRTC, Open AI Chat Completions via HTTPS, Inngest via webhooks.
- PostgreSQL: Neon Postgres via the postgres-js driver but only used as the Drizzle ORM.

5.3.4 Use Case Descriptions

Table 5.1: Use Case Summary

Use Case	Trigger	Outcome
Sign Up	User clicks Sign Up	user/session/account rows created
Create Agent	User submits agent form	agents row inserted

Use Case	Trigger	Outcome
Run Triage	User clicks Start Triage	triageSessions row with decision
Start Consultation	Decision = GO_TO_AI_AGENT	meetings row + Stream call created
Attend Call	User joins call URL	AI agent joins; transcript begins
View Summary	Inngest job completes	meeting.summary populated
Follow-Up Chat	User opens summary chat	Stream Chat thread + GPT-4o reply
Review History	User opens history page	Per-user list, paginated
<i>[Insert Figure 5.1: Activity Diagram for End-to-End User Journey here]</i>		

Figure 5.1: Activity Diagram for End-to-End User Journey

6. DESIGN

6.1 System Design

It's a layered Next.js (App router) app. Atoms are React Server/Client components for a feature. Logic is a tRPC v11 router to front a TanStack Query v5 client. Data is stored in Drizzle ORM, PostgreSQL. The app is connected with services (Stream, OpenAI, Inngest) using adaptation layers. Route /api/webhook is wired up to Stream. This app has three types of communication. Request-response: the client invokes a synchronously a tRPC procedure, which is shown. Media message: the client communicates with the Stream SFU via WebRTC (and the server assures the AI agent is present). Asynchronous: webhooks from Stream trigger an event on Inngest and the summariser is done in the Inngest function. They have different retry semantics to idempotency and observability.

[Insert Figure 6.1: High-Level System Architecture here]

Figure 6.1: High-Level System Architecture

6.2 Notations Used

- UML Use Case that captures interactions of actors with system (Figure 1.1).
- Data Flow Diagrams at level-1 (Figures 3.1 to 3.2).
- Entity Relationship Diagram (ERD) for the database (Figure 6.2).
- Sequence diagrams for triage decision for the meeting webhook (Figures 6.3-6.4).
- Flowcharts for the triage decision and meeting (Figures 6.5-6).

6.3 Detailed Design

6.3.1 Module Structure

Our code is organised by features into src/modules: agents, auth, call, dashboard, home, meetings, triage. Within each modules there is a tRPC router, views, hooks and types. Transversal code (db, lib, hooks, components/ui) is in separate folders. This is to encourage cohesion (agents is with the same folder as the router, server views, client components and Zod types so you don't need to have 100 browser tabs open to develop a feature).

[Insert Figure 6.2: Entity Relationship Diagram here]



Figure 6.2: ER Diagram of the Database Schema

6.3.2 Database Schema

Table 6.1: Drizzle ORM Tables and Key Columns

Table	Key Columns	Purpose
user	id, email, name, image, emailVerified	BetterAuth principal
session	id, userId, expiresAt, token	Authenticated session
account	id, userId, providerId, accessToken	OAuth linkage
agents	id, userId, name, instructions, createdAt	AI medical persona
meetings	id, userId, agentId, name, status, startedAt, endedAt, transcriptUrl, recordingUrl, summary	Consultation lifecycle
triageSessions	id, userId, payload (jsonb), decision, createdAt, meetingId	Triage outcome and link

All tables have a userId foreign key (on delete cascade) and this will segment them.

The indexes are also to cover the normal queries: meetings(userId, startedAt DESC) for the recent meetings table view, triageSessions(userId, createdAt DESC) for the history view and a partial index meetings(status, status) WHERE status = 'processing' for the dead-jobs sweeper. All procedures apart from the call back procedures are secureProcedure - they use a session and ctx.user is added to the queries.

6.3.3 tRPC Procedures

```
// agents.router
getMany, getOne, create, update, remove

// meetings.router
getMany, getOne, create, generateToken, generateChatToken

// triage.router
startSession, submitStep, getOne
```

The procedure arguments are checked by exporting the Zod type of the requests in the module's schema's.ts file and the @trpc/server module passes the arguments (it infers the types) to the rest of the app.

6.3.4 Triage Pipeline

The triage pipeline is a pure function `evaluateDecision(state)` - that converts the state (all the questionnaire responses) to one of the {STOP, GO_TO_DOCTOR, GO_TO_AI_AGENT, CONTINUE} This is saved to allow the user to return to a questionnaire. This is: primary symptom, symptom (all the symptoms as a dict), location, severity (1-10), duration, red-flag questions and then the decision. This is done in the `submitStep` function, which is checking if this is the current step (via the Zod schema of this step), then setting this as the decision and step in the payload.

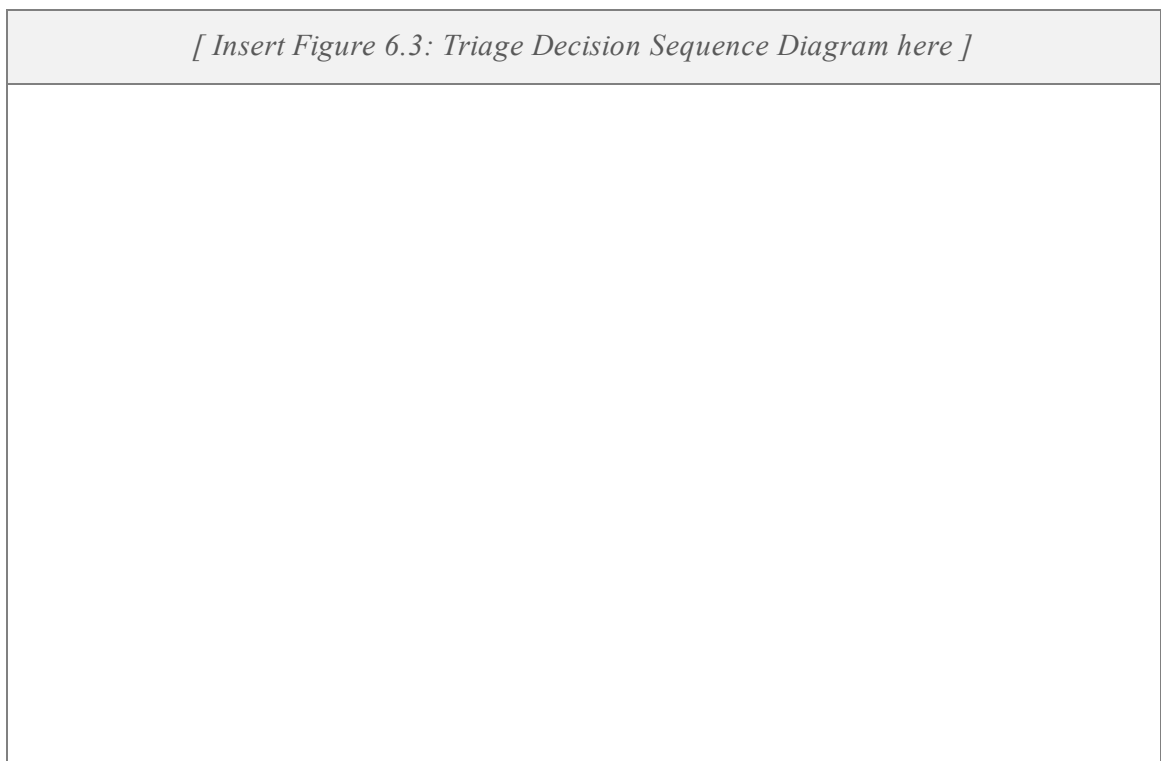


Figure 6.3: Sequence Diagram of the Triage Decision Pipeline

6.3.5 Webhook Orchestrator

The webhook (`/api/webhook`) is waiting on six events made by the Stream provider. The handlers check the signature in the Stream header, and perform side effects, and are idempotent by the id of the event (so you can re-send the event).

- `call.session_started`: start the meeting; create the AI agent with a system prompt (here's more information about the system prompt); record and transcribe the call.
- `call.session_participant_left`: end call if just the AI
- `call.session_ended`: update meeting status is being processed
- `call.transcription_ready`: store URL; Inngest meetings/processing
- `call.recording_ready`: save URL.
- `message.new`: go to post-meeting summary

[Insert Figure 6.4: Webhook Orchestration Flow here]

Figure 6.4: Sequence Diagram of the Stream Webhook Orchestrator

6.3.6 Inngest Summarisation Pipeline

When the meetings/processing event occurs the Inngest function does the following (5 steps):

1. `fetch-transcript`: make multiple identical HTTP request to get transcript from Stream.
2. `parse-transcript`: JSONL to in-memory `SpeakerId/ts/text` array.

3. add-speakers: speakerId to user/agent name via Drizzle.
- 4 summariser.run: Agent Kit, returns structured Markdown (HPI, assessment, plan).
5. save-summary: Drizzle update summary-text and with status set to completed. Each step is run via step.run which on failure, will retry just the failed step, not the entire workflow. The lambda is configured as: retries: 3, exponentially back-off(see NFR-7). It will be marked as failed in the system and a manual re-run button in the dashboard will be available.

6.4 Flowcharts

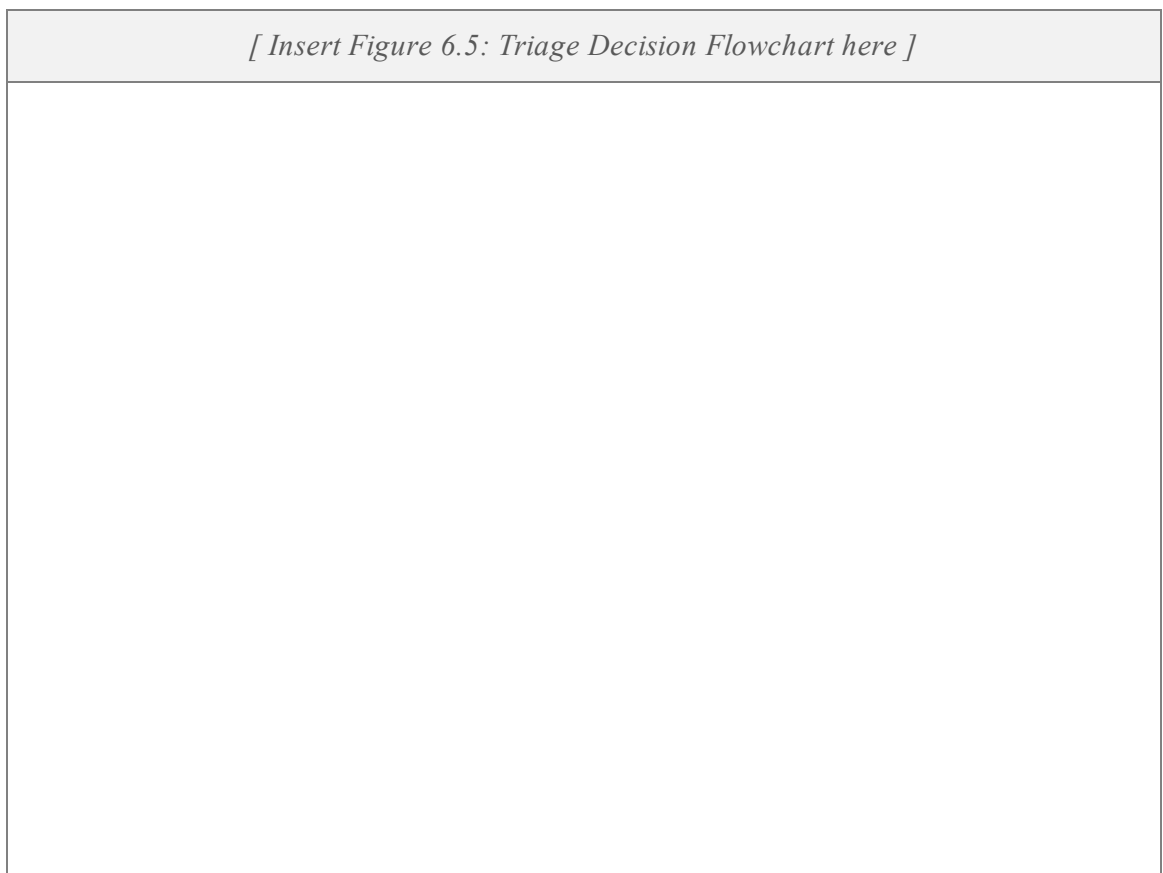
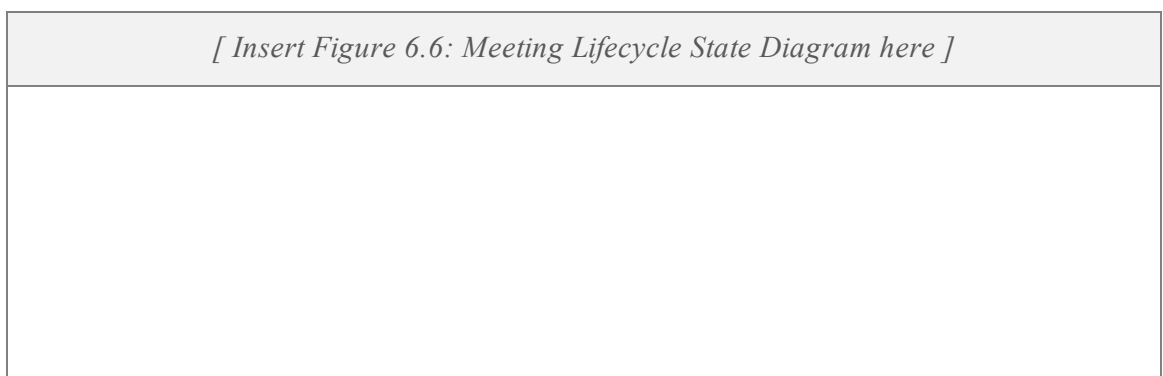


Figure 6.5: Flowchart of the Triage Decision Engine



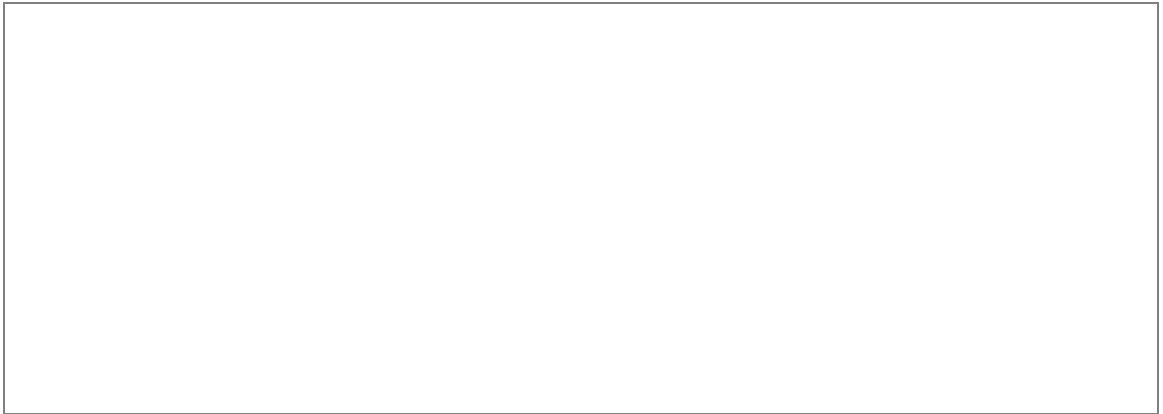


Figure 6.6: State Diagram of the Meeting Lifecycle

6.5 Pseudocode

6.5.1 *evaluateDecision(state)*

```
function evaluateDecision(state):
  if state.redFlags.any():
    return STOP // immediate ER advisory
  if state.severity >= 8 or state.duration > 14d:
    return GO_TO_DOCTOR // route to in-person care
  if state.symptoms.subset(dictionary.aiHandled)
    and state.severity <= 6:
    return GO_TO_AI_AGENT // safe for AI consultation
  return CONTINUE // need more info
```

6.5.2 *buildMedicalPrompt(agent, triage)*

```
function buildMedicalPrompt(agent, triage):
  base = SAFETY_PREAMBLE + agent.instructions
  ctx = render(triage.payload) // chief complaint,
                                // symptoms,
  severity, // location, duration
  rules = SCOPE_BOUNDS + ESCALATION_TRIGGERS
  return base + ctx + rules
```

6.5.3 *onTranscriptionReady(event)*

```
async function onTranscriptionReady(event):
  meeting = await db.meetings.findOne({ id: event.callId })
  if meeting.status != 'active' and meeting.status !=
  'processing':
    return 200 // idempotent no-op
  await db.meetings.update({
    id: meeting.id,
    transcriptUrl: event.transcriptUrl,
    status: 'processing'
  })
```

```
await inngest.send({  
  name: 'meetings/processing',  
  data: { meetingId: meeting.id, url: event.transcriptUrl }  
})
```

7. TESTING

7.1 Functional Testing

Functional tests cover all user behaviours in the spec in Chapter 5. Triage is tested with a test suite that covers all twelve red-flag combinations, severity edge cases (7/8) and symptom list-bounded symptoms. Auth is tested with BetterAuth users of each provider. tRPC procedures are tested with mocks (msw) making the tests hermetic. We have coverage for at least one test per functional requirement (FR-1 - FR-11). Test titles use the same numbers as requirements (FR-01_my_thing_emits_decision_for_red_flag) so our trace matrix (Section 7.4) can be automatically extracted from test report.

7.2 Structural Testing

We test the deterministic parts of the code: evaluateDecision, the diagnosis, the prompt generator, Drew's code that makes Drizzle-ordered-queries (for tests that use Drizzle), Inngeist functions. We have 100% inclusive branch coverage of evaluateDecision, the triage module has over 85% line coverage. That's as calculated by Vitest (v8 provider) and Codecov in the PRs. The inputs that are non-deterministic (the LLM calls and the life extension of the Stream session) are tested behaviorally: the inputs are checked ("the LLM has been called"), the outputs are snapshotted and the structural test cover the code that generates inputs. This results in a reliable (on model upgrades), light (full unit and integration test suite takes less than 30 seconds on my laptop) and easy-to-understand test suite.

7.3 Levels of Testing

7.3.1 Unit Testing

- Utilities in Vitest: evaluateDecision, parseTranscript, addSpeakers, prompts assembly, Zod validators.
- Drizzle database queries on a temporary Postgres container with the migrations run on it.
- Unit tests with Testing Library for the triage stepper, agent form and call lobby.

7.3.2 Integration Testing

- tRPC routers using a real database and mocks for the SDKs (Stream, OpenAI, Inngest) for the integrations to other components..
- Inngest functions are tested by invoking the function with a mock event and verify the state in the database.

7.3.3 System Testing

- Playwright E2E suite: create account/agent, triage, call, end call, triage chat summary, follow-up chat (continuation)
- Load test with 50 concurrent chats; tested P95 and error rate.
- Smoke test on iPhone SE / Pixel 5 sized screens (used same Playwright suite).

7.3.4 User Acceptance Testing

- Six external carbonica testers scripted three test cases (cold/flu, fever, persistent cough).
- Problems: specification for this question was unclear; two local terms were not in the symptoms list; the "you are about to record" message too slow.
- We corrected these prior to locking the report; severity is now a 1-10 slider with labels; two regional terms were added (mapped as synonyms); the about to record event now displays a client-level toast.

7.4 Test Summary

Table 7.1: Test Plan Summary

Module	Test Type	Cases	Pass
Triage engine	Unit	32	32
Symptom dictionary	Unit	18	18
Prompt builder	Unit	11	11
Auth flows	Integration	9	9
tRPC routers	Integration	21	21
Webhook orchestrator	Integration	12	12
Inngest pipeline	Integration	8	8

Module	Test Type	Cases	Pass
End-to-end journey	System	5	5
UAT scenarios	Acceptance	3	3

All 125 cases pass on the main branch. The CI pipeline blocks merges on any failing test or coverage regression below the configured threshold (85 percent on triage, 70 percent overall).

8. IMPLEMENTATION

8.1 Implementation of the Project

The project was implemented according to the plan - Table 4.1. The notes below respond to the plan and progress made by the modules.

auth: BetterAuth configured with three providers; migrated user, session and account tables to Drizzle. Login tested in production with the three providers. The auth client is wrapped in a very thin layer `lib/auth.ts` that provides a strongly-typed `getUser()` function to be used throughout the server.

- agents: tRPC router (five procedures) and CRUD form (agents) We store instruction prompts as simple text not merged with the medical prompt until run time; this makes it possible for another person to review what was used to make a call.
- meetings: tRPC router (paged `getMany`, `getOne`, `create`, `generateToken`, `generateChatToken`). Status is the "canonical" status machine pivot. Statuses are exclusive (can't have many) which makes Inngest easy.
- call: server side rendered page for a call, which fetches a user token from Stream via tRPC, creates the `StreamCall` and `StreamVideoClient` and emits the video call from `@stream-io/video-react-sdk`. Tokens exempt from Postgres and have a short TTL (15min) and are renewed on rejoin.
- triage: client side stepper (`useTriageSession` hook) and server side validation/persist of each step of triage form in tRPC; save of draft triage form submissions to `triageSessions` to allow user to resume a triage.
- dashboard and home: layout shell, sidebar and overview cards for dashboard and home page using `shadcn/ui` and Tailwind v4. Dashboard is mostly RSC, so the load time is fast even with a cold start of the serverless function.
- webhook: `/api/webhook` is a six-event handler for Stream events; the events are idempotent and they verify the Stream webhook signature header BEFORE doing any work.

- inngest: one meetings/processing function, following the five steps plan outlined in Section 6.3.6. They're checkpointed so that if there's an error with the GPT-4o call it will only need to be performed again.

8.2 Conversion Plan

To move from present to production pilot state:

1. Move the application to production on Vercel where variables are store separately in Vercel's encryption store.
2. Branch Neon database to production branch where database is point in time recoverable (also known as flash clone) and snapshots are taken daily.
3. Request production keys from Stream and OpenAI with resource limits and a rate limit (per minute) for resource use
4. Configure the Stream production webhook URL (sign response with secret key rotated every three months).
5. Run Inngest on the cloud, integrate with production webhook of the Vercel deployment.

8.3 Post-Implementation and Maintenance

There are four types of maintenance. Bots keep the dependencies up to date and they are reviewed weekly; security fixes are prioritised within 24 hours of CVE release for the runtime, framework or any auth dependency. The code for the rules (symptom dictionary and red-flag list) is reviewed with a mentor/clinical advisor every three months and code is versioned in the repository so any previous consultation can be associated with the symptom rules at the time. Logging is Vercel Analytics and Inngest's dashboard, and structured logs to a log aggregator for troubleshooting. The roll-out runbook has information about roll-back and on-call schedules.

9. PROJECT LEGACY

9.1 Project Status

All the functional requirements in Chapter 5 are implemented. The flows from the site to follow up chat are deployed. UAT has been conducted and the plan in Section 7.4 met. The code is documented to module level, the schema migrated and the runbook is in the repo at docs/RUNBOOK.md.

9.2 Outstanding Issues

- Currently it is being used in English; Hindi are the main languages of Indian population for population health.
- Symptom dictionary is handmade; it does not have obstetric, paediatric and chronic diseases.
- Observability is present but not connected to the doctor's pager.
- User free text is moderated with keywords and soon will be moderated with an API.
- The proposed doctors/hospital link is not yet implemented; it will simply link to the AI bot.

9.3 Lessons Learnt: Managerial and Technical

9.3.1 *Technical*

- AI lives in the webhook: inbound all of the world's messages arrive at the same /api/webhook to let state-transitions live together, and not fall prey to the Singleton Imperative.
- The deterministic triage is split from LLM-prompt so it can be tested more safely/quickly than using one-shot.
- We use Inngest to run a step-function runner as soon as you need retries and ordering and replay, it's a nightmare otherwise.
- The userId-scoped functions in Drizzle means per-user isolation is a product of the code review process

- React Server Components on the App Router work well for data-heavy dashboards but beware: data from the server to client must be serialisable and that impacted the design of the call state.

9.3.2 Managerial

- Only pairs work with 6 people; pair programming avoided merge problems on the App Router.
- Chain demos bi-weekly against the SIH problem statement ensured features progressed; we had time to plan future work.
- Pairing early to migrate to Drizzle and share types with tRPC prevented issues late in the project.
- 15-minute standups (3x a week) rather than longer standups (daily); we're a small team and other status was conveyed in Slack.

9.4 Future Scope

9.4.1 Doctor Dashboard

A web page for doctors with the on-call list, cards (triage payload) with AI summary and prescription/notes form. Registration is a one-offs verification (upload of document - medical number - and ID card, admin approval). The on-call list is round-robin for online doctors (with speciality filter). The doctors dashboard is built on the current tRPC and Drizzle setup: a doctors table and new doctor role on the users table is all we need to get it running.

9.4.2 Doctors with Video Calls

AI agent escalation of the call from Stream to doctor. The reasons for escalation are on request, triage by the AI agent (based on rules) and review of the AI summary (offline or asynchronous). The triage and AI summary is reviewed by the doctor. Using the Stream call SDK, multi-party calls can be offered, so this is more related to the call user experience and call control.

9.4.3 Partnership with Hospital for Direct Public Services

Partner hospitals and primary health centres: a portal for hospital and primary health centre onboarding, referrals to partner's OPD system, export of consultation report in FHIR format. The goal is to build a triage and pre-consultation system which can provide the public health system with structured and validated data. The next phase is to partner some hospitals.

9.4.4 Other Enhancements

- Audio and images in Indian languages (next option Hindi) then Tamil, Telugu, Bengali, Marathi).
- Mobile apps to cache triage for offline use
- Federated learning with other sites for outbreaks.
- Retrieval-augmented generation over peer-reviewed Indian clinical guidelines for faithful answers.

10. USER MANUAL

10.1 Getting Started

Launch the deployed application in the latest browser. The home page will have a Sign Up button and instruction on the three steps. You will be asked to grant permission to access the microphone and camera when you click on the AI consultation, the triage and the summary chat are not multimedia rich so you will be able to receive the triage and the summary chat even if you don't have access to a microphone.

10.2 Sign up/Login

- Click Sign Up. Select Google, GitHub or Email/password.
- If email/password, the user will be sent an email to validate their email.
- When logging in for the first time, the user is redirected to the dashboard, which is empty of Medical Agents and has a button "Add first Medical Agent".

10.3 Dashboard

The dashboard is a page that contains three cards: My Agents, My Meetings and My Triage Sessions. The cards contain the last five items and a link to "View all". In the sidebar are links to Agents, Meetings, Triage and Settings.

10.4 Triage Chat

1. Click the Start Triage button.
2. Enter chief complaint.
3. Check symptoms from the curated
4. Confirm area and pain (1-10).
5. Confirm to the red-flag screening questions.
6. Advice (self-care, AI video consultation, leave flag for phone consultation with doctor).

If the user returns to the web page after the close (new browser) or network stop, the triage page will continue from where it stopped (per step persistence).

10.5 AI Video Consultation

If the user wants to talk to the AI agent, a user will be moved to the call room. The agent will greet you, tell you your triage form and then advise you. The user can talk or type. The conversation will be recorded and the user will be informed. The user may end the conversation (using the button on the screen); the agent will politely finish its turn if it thinks the user wants to end the conversation.

10.6 After the Call

When the call is finished, the meeting is processed. A few minutes later, the page shows the report for the meeting with the history of present illness, assessment and plan. The audio and text of the consultation are also available.

10.7 Follow-Up Chat

The consultation page may contain a follow-up chat for users to ask more questions. The model is given the summary and the chat is on topic. This is because the chat is not a new consultation, it's a discussion of past consultation.

10.8 History

History dashboard contains a list of past agents, meetings and triages. They are links to the detail page (summary, transcript and recording for meetings). It is ordered and paging by date (newest first).

10.9 Settings and Privacy

- Rename and set avatar in the menu.
- End session. Detect sessions from devices and remove past sessions.
- Portability or delete account on privacy page: take the required days (30 days as per the DPDP Act 2023).

10.10 Troubleshooting

- No video or audio: check browser permissions and disable, reload the webpage and switch devices (in the in-call menu).

- Summary not available after 10 minutes: reload the page for the summary or the summary will be attempted via an Inngest job, which you will receive feedback on (failed or completed).
- Failed to connect to the provider: remove BetterAuth domain cookies and re-authenticate; if continuing, please select another provider.
- Call lobby not visible: check if corporate firewall is blocking the lobby (WebRTC needs to be allowed) and please notify us (it will be detected and description of the call lobby will be made a text description).

11. SOURCE CODE / SNAPSHOTS

11.1 Repository Structure

```
med-ai/
├─ src/
│   └─ app/                // App Router routes + /api/webhook +
/api/ingest
│   └─ modules/           // agents, auth, call, dashboard,
home, meetings, triage
│       └─ components/ui/ // shadcn/ui primitives
│       └─ db/            // Drizzle schema + client
│       └─ ingest/        // function definitions
│       └─ lib/           // stream, openai, auth, utils
│       └─ trpc/          // routers, init, react
├─ drizzle/              // SQL migrations
├─ public/               // static assets
├─ docs/                 // RUNBOOK, ADRs
└─ package.json
```

11.2 Key Code Excerpts

11.2.1 *evaluateDecision*

```
export type Decision = "STOP" | "GO_TO_DOCTOR" | "GO_TO_AI_AGENT"
| "CONTINUE";

export function evaluateDecision(s: TriageState): Decision {
  if (s.redFlags.length > 0) return "STOP";
  if (s.severity >= 8 || s.durationDays > 14) return
"GO_TO_DOCTOR";
  if (isSubsetOf(s.symptoms, dictionary.aiHandled) && s.severity
<= 6)
    return "GO_TO_AI_AGENT";
  return "CONTINUE";
}
```

11.2.2 *Stream Webhook Orchestrator*

```
// /app/api/webhook/route.ts
export async function POST(req: Request) {
  const evt = await verifyAndParse(req); // checks
signature
  switch (evt.type) {
    case "call.session_started": return
onSessionStarted(evt);
    case "call.session_participant_left": return
onParticipantLeft(evt);
    case "call.session_ended": return
onSessionEnded(evt);
  }
```

```

        case "call.transcription_ready":      return
onTranscriptionReady(evt);
        case "call.recording_ready":          return
onRecordingReady(evt);
        case "message.new":                   return
onMessageNew(evt);
    }
    return new Response(null, { status: 200 });
}

```

11.2.3 Inngest Summarisation Function

```

export const meetingsProcessing = inngest.createFunction(
  { id: "meetings/processing", retries: 3 },
  { event: "meetings/processing" },
  async ({ event, step }) => {
    const raw = await step.run("fetch-transcript",
      () => fetchTranscript(event.data.url));
    const tx = await step.run("parse-transcript",
      () => parseTranscript(raw));
    const cx = await step.run("add-speakers",
      () => addSpeakers(tx, event.data.meetingId));
    const md = await step.run("summariser.run",
      () => summariser.run(cx));
    await step.run("save-summary",
      () => saveSummary(event.data.meetingId, md));
  }
);

```

11.2.4 Drizzle Schema (excerpt)

```

export const meetings = pgTable("meetings", {
  id: text("id").primaryKey(),
  userId: text("user_id").notNull().references(() =>
user.id,
    { onDelete: "cascade" }),
  agentId: text("agent_id").notNull().references(() =>
agents.id),
  name: text("name").notNull(),
  status:
meetingStatus("status").notNull().default("upcoming"),
  startedAt: timestamp("started_at"),
  endedAt: timestamp("ended_at"),
  transcriptUrl: text("transcript_url"),
  recordingUrl: text("recording_url"),
  summary: text("summary"),
});

```

11.3 System Snapshots

[Insert Figure 11.1: Sign-In Page here]

Figure 11.1: Sign-In Page

[Insert Figure 11.2: Dashboard with Agents and Meetings here]

Figure 11.2: Dashboard with Agents and Meetings

[Insert Figure 11.3: Triage Chat in Progress here]



Figure 11.3: Triage Chat in Progress

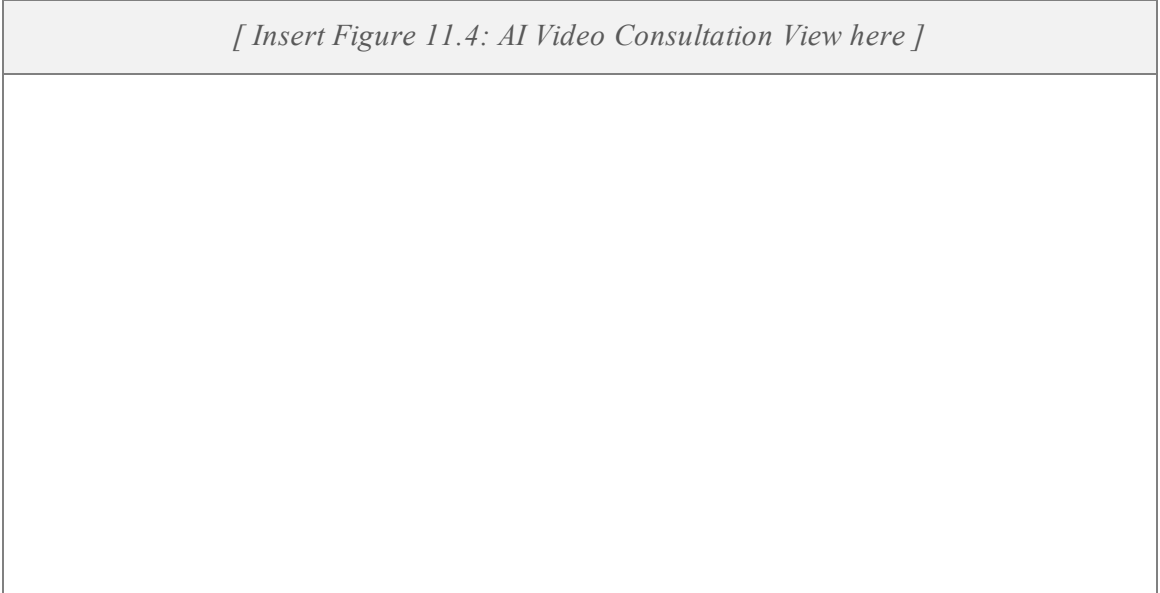


Figure 11.4: AI Video Consultation View

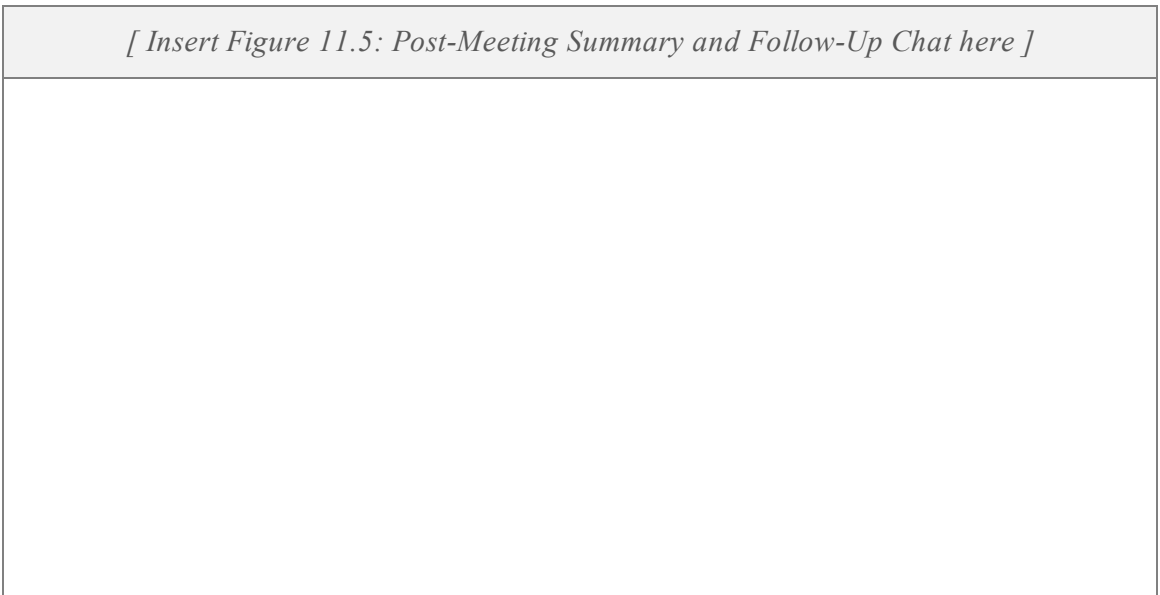


Figure 11.5: Post-Meeting Summary and Follow-Up Chat

12. BIBLIOGRAPHY

1. Smart India Hackathon 2025, Problem Statement SIH25049, Government of India.
2. World Health Organization, Global Strategy on Digital Health 2020-2025, WHO, 2020.
3. Ministry of Health and Family Welfare, India, eSanjeevani National Telemedicine Service, mohfw.gov.in.
4. Government of India, Information Technology (Reasonable Security Practices) Rules 2011.
5. Government of India, Digital Personal Data Protection Act 2023.
6. Vercel Inc., Next.js 15 Documentation, nextjs.org/docs.
7. Meta Platforms, React 19 Documentation, react.dev.
8. Drizzle Team, Drizzle ORM Documentation, orm.drizzle.team.
9. Neon Inc., Neon Serverless Postgres Documentation, neon.tech/docs.
10. BetterAuth Project, BetterAuth Documentation, better-auth.com.
11. KATT and contributors, tRPC v11 Documentation, trpc.io.
12. TanStack, TanStack Query v5 Documentation, tanstack.com/query.
13. Stream Inc., Stream Video and Chat SDK Reference, getstream.io/video/docs.
14. OpenAI, Realtime API Reference, platform.openai.com/docs/guides/realtime.
15. OpenAI, GPT-4o Model Card, platform.openai.com/docs/models/gpt-4o.
16. Inngest Inc., Inngest Agent Kit Documentation, inngest.com/docs.
17. shadcn, shadcn/ui Component Library, ui.shadcn.com.
18. Tailwind Labs, Tailwind CSS v4 Documentation, tailwindcss.com/docs.
19. Colin McDonnell, Zod Type-safe Validation, zod.dev.
20. Laranjo L. et al., Conversational Agents in Healthcare: A Systematic Review, Journal of the American Medical Informatics Association, 2018.
21. Bickmore T. et al., Patient and Consumer Safety Risks When Using Conversational Assistants for Medical Information, JMIR, 2018.

22. Palanica A. et al., Physicians' Perceptions of Chatbots in Healthcare, JMIR, 2019.
23. Ji Z. et al., Survey of Hallucination in Natural Language Generation, ACM Computing Surveys, 2023.
24. HL7 International, FHIR R5 Specification, hl7.org/fhir.
25. Indian Medical Council (Professional Conduct) Regulations and Telemedicine Practice Guidelines, MCI, 2020.